

Doc. No.: WG21/P0250R0

Date: 2016-02-12

Author: Hans-J. Boehm

Email: hboehm@google.com

P0250R0: Wording improvements for initialization and thread ids (CWG 2046)

This is an attempt to turn the Kona SG1 discussion of CWG 2046 into wording. Unfortunately, that discussion exposed a number of related issues.

In several places the standard needs to require that some operation happens before another one, in roughly, but not quite, the sense of the 1.10 memory model. We expect this happens before relation to compose with sequencing. But our normal happens before relation no longer composes in this way, due to the presence of `memory_order_consume`. Hence we introduce the required notion in 1.10.

Also note that sequencing is intended to be a relation on evaluations within a single thread. It somehow had been abused for other things in the context of static initialization, which was never intended to be confined to a single thread.

The current version is incomplete. In particular it does not yet fully address the thread id validity issues. We want to require that main, atexit handlers, and constructors/destructors run in a proper thread with a proper (not default constructed) thread id.

SG1 also concluded that a C++ program should be allowed to construct multiple threads for constructors etc., even if the program itself did not use threads. That is also not reflected here. This "decision" may be worth revisiting since it technically breaks compatibility with C++03.

Wording changes

(Fixes an editorial ugliness that somehow crept in.) Move 1.9p6, which uses sequenced before, to someplace below the definition of sequenced before in 1.9p13 [intro.execution].

Add after 1.10p14 [intro.multithread]:

An evaluation A *strongly happens before* an evaluation B if either

- A is sequenced before B, or
- A synchronizes with B, or
- A strongly happens before X and X strongly happens before B.

[Note: In the absence of consume operations the happens before and strong happens before relations are identical. Strong happens before essentially excludes consume operations.]

Change the last part of 3.6.2p2 [basic.start.static] as follows:

If constant initialization is not performed, a variable with static storage duration (3.7.1) or thread storage duration (3.7.2) is zero-initialized (8.5). Together, zero-initialization and constant initialization are called static initialization; all other initialization is dynamic initialization. Within each phase, static initialization shall be performed strongly happen before (1.10) any dynamic initialization takes place. [Note: The dynamic initialization of non-local variables is described in 3.6.3; that of local static variables is described in 6.7. — end note]

Change 3.6.3p1 [basic.start.dynamic] as follows:

Dynamic initialization of a non-local variable with static storage duration is unordered if the variable is an implicitly or explicitly instantiated specialization, and otherwise is ordered [Note: an explicitly specialized static data member or variable template specialization has ordered initialization. — end note]. Variables with ordered initialization defined within a single translation unit shall be initialized in the order of their definitions such that initialization corresponding to earlier definitions strongly happen before those for later definitions in the translation unit. If a program starts a thread (30.3), the subsequent any initialization not specified to happen before the first thread creation may be performed by either the main thread or a thread implicitly created to perform initializations. If such initializations are unordered or defined in separate translation units, no happens before ordering is specified between them. of a variable is unsequenced with respect to the initialization of a variable defined in a different translation unit. Otherwise, the initialization of a variable is indeterminately sequenced with respect to the

initialization of a variable defined in a different translation unit. If a program starts a thread, the subsequent ~~unordered initialization of a variable is unsequenced with respect to every other dynamic initialization. Otherwise, If two unordered initializations happen before the start of the first additional thread, or the program starts no additional threads, then those~~ the unordered initializations of a variable ~~is~~ are indeterminately sequenced with respect to ~~each other every other dynamic initialization~~. [Note: This definition permits initialization of a sequence of ordered variables concurrently with another sequence. — end note]

3.6.3p2 appears to have some unfortunate interactions with signal handlers. We may want to address those here, but they currently are not addressed. P0270R0 has more details.

Change the beginning of 3.6.3p2 as follows:

It is implementation-defined whether the dynamic initialization of a non-local variable with static storage duration strongly happens before the first statement of main. If the initialization is deferred to some point in time after the first statement of main, it strongly happens before the first odr-use (3.2) of any function or variable defined in the same translation unit as the variable to be initialized.

Change 3.6.3p3 as follows to fix what looks like an unrelated bug:

It is implementation-defined whether the dynamic initialization of a non-local variable with ~~static or~~ thread storage duration is sequenced before the first statement of the initial function of the thread. If the initialization is deferred to some point in time after the first statement of the initial function of the thread, it is sequenced before the first odr-use (3.2) of any variable with thread storage duration defined in the same translation unit as the variable to be initialized.

Change 3.6.4p1 [basic.start.term] as follows:

Destructors (12.4) for initialized objects (that is, objects whose lifetime (3.8) has begun) with static storage duration are called as a result of returning from main and as a result of calling `std::exit` (18.5). The return from main or the call to `std::exit` respectively strongly happen before the destructor invocations.

Destructors for initialized objects with thread storage duration within a given thread are called as a result of returning from the initial function of that thread and as a result of that thread calling `std::exit`. The completions of the destructors for all initialized objects with thread storage duration within that thread ~~are sequenced~~ strongly happen before the initiation of the destructors of any object with static storage duration. If the completion of the constructor or dynamic initialization of an object with thread storage duration is sequenced before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first.

If the completion of the constructor or dynamic initialization of an object with static storage duration ~~is sequenced~~ strongly happens before that of another, the completion of the destructor of the second ~~is sequenced~~ strongly happens before the initiation of the destructor of the first. [Note: This definition permits concurrent destruction. — end note] If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. For an object of array or class type, all subobjects of that object are destroyed before any block-scope object with static storage duration initialized during the construction of the subobjects is destroyed. If the destruction of an object with static or thread storage duration exits via an exception, `std::terminate` is called (15.5.1).

Change 3.6.4p3 [basic.start.term] as follows:

If the completion of the initialization of an object with static storage duration ~~is sequenced~~ strongly happens before a call to `std::atexit` (see <cstdlib>, 18.5), the call to the function passed to `std::atexit` ~~is sequenced~~ strongly happens before the call to the destructor for the object. If a call to `std::atexit` ~~is sequenced~~ strongly happens before the completion of the initialization of an object with static storage duration, the call to the destructor for the object ~~is sequenced~~ strongly happens before the call to the function passed to `std::atexit`. If a call to `std::atexit` ~~is sequenced~~ strongly happens before another call to `std::atexit`, the call to the function passed to the second `std::atexit` call ~~is sequenced~~ strongly happens before the call to the function passed to the first `std::atexit` call.